SnapDocc : Project Documentation

SnapDocc is a WhatsApp bot that turns a photo of a list, receipt, table or handwritten note into an Excel, Word, PDF, CSV or TXT file, sent back in the same chat.

Overview

SnapDocc lets a person digitize paper without leaving WhatsApp: send a photo, choose whether to edit the text first, and receive the finished file in the same chat.

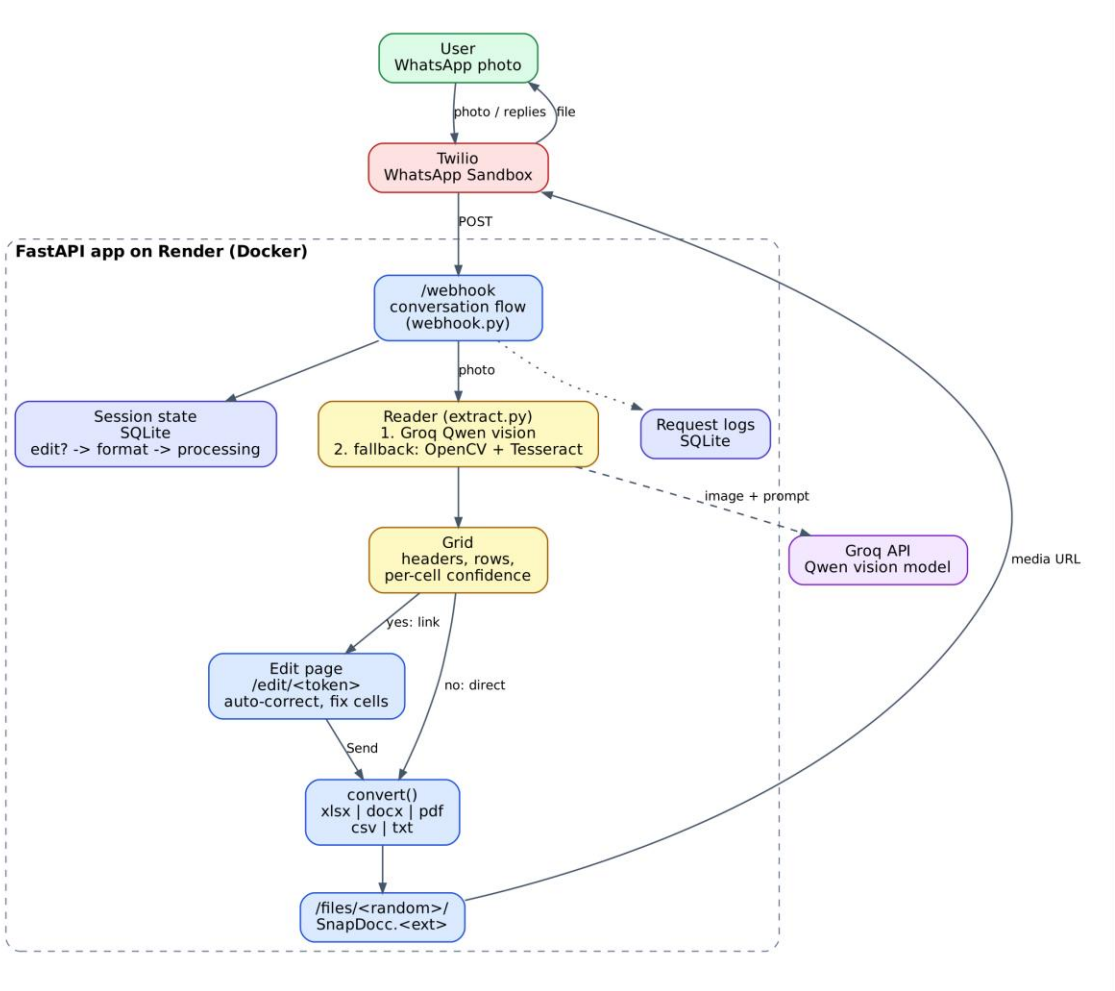
- **Goal:** turn handwritten and printed documents into structured files with as little typing as possible.
- **Reader:** Qwen vision model on Groq (primary), OpenCV + Tesseract (automatic fallback).
- **Outputs:** Excel (.xlsx), Word (.docx), PDF, CSV, TXT.
- **Live at:** https://snapdocc-web-app.onrender.com (Twilio WhatsApp sandbox, Render free tier).
- **Stack:** Python, FastAPI, Uvicorn, Twilio, Groq API, OpenCV, Tesseract, openpyxl, python-docx, reportlab, SQLite with SQLAlchemy, Docker, Render.

Requirements

Requirement	How it is met
Conversation flow	Photo is saved and a session stored. The bot asks "edit first?", then the format. Invalid replies are re-asked and sessions expire.
OCR module	Groq Qwen vision (primary), OpenCV + Tesseract (fallback), EasyOCR for evaluation. Returns text, boxes and per-word confidence.
Structuring	Detects list or table from word positions. Lists are split into quantity, unit and item.
File generators	One function per format behind `convert(data, fmt)`. Low-confidence cells are highlighted yellow in Excel.
Logging	Every request stores time, document type, format, engine, average confidence, seconds and status.
Evaluation	`eval/evaluate.py` runs the engines and prints a CER and WER comparison table.
Error handling	5 MB size limit, non-image files rejected, empty reads reported. Blurry photos are still processed; unsure cells are flagged instead of asking to resend.

Architecture

A photo travels from the user's phone to Twilio, then to the FastAPI webhook on Render. The reader turns it into a table, and the generators turn the table into a file that Twilio delivers back to the chat.



1. The user sends a photo. Twilio posts it to `/webhook`.
2. `webhook.py` downloads and checks the image, saves a session, and asks "edit first?".
3. The reader (`ocr/extract.py`) asks Groq Qwen vision for a table. If that fails it uses OpenCV and Tesseract and says so.
4. The result is a grid of headers, rows and a confidence value per cell.
5. Edit path: the bot sends an edit link, and the page sends the file back. Direct path: `convert()` builds the file immediately.
6. The file is served from `/files/<random-folder>/` and Twilio delivers it to WhatsApp.

Modules

Each folder has one job, which keeps the code easy to test and to explain.

Module	What it does
`app/main.py`	FastAPI app, the `/webhook` route, serves generated files at `/files`, runs cleanup at startup
`app/webhook.py`	The conversation: photo, edit question, format question, then the job

Module	What it does
`app/session.py`	Session state in SQLite with an expiry time
`app/messaging.py`	Sends WhatsApp messages through Twilio; queues a message when Twilio refuses it and hands it over on the user's next reply
`app/web.py`	Edit page: shows photo and table, auto-correct, send to WhatsApp
`app/ocr/`	Groq Qwen reader, Tesseract and EasyOCR engines, image preprocessing, `extract.py` (picks the engine), `groq_check.py` (test tool)
`app/structuring/`	Detects list or table from word positions, parses rows, columns and quantity / unit / item
`app/generators/`	`convert (data, fmt)` and one generator per format, plus cell sanitizing for formulas
`app/utils/`	Grid helpers, Groq client, spelling and grammar auto-correct, format parsing, CER and WER metrics
`app/db/`	SQLAlchemy models (sessions, edit jobs, request logs) and a log viewer
`app/housekeeping.py`	Deletes old photos, files, links and sessions
`eval/`	Sample generator and the CER / WER comparison script
`tests/`	85 pytest tests with Twilio and Groq mocked

Conversation flow

The bot keeps one session per phone number. The session state decides how the next message is read.

State	Bot asks	Valid replies	Next state
(none)	Welcome text	A photo	`awaiting_edit_choice`
`awaiting_edit_choice`	"Do you want to review and edit first?"	1 / yes (edit), 2 / no (direct), or a format name	`processing` or `awaiting_format`
`awaiting_format`	"Which format do you want?"	1 to 5, or a format name	`processing`
`processing`	"Still reading your photo..."	Nothing needed	Session cleared when the job ends

Rules that apply everywhere:

- Every photo is accepted on the first send. The bot never asks the user to resend.
- Invalid replies get "I didn't understand" plus the question again.
- Sessions expire after a timeout, so an old photo is never answered by mistake.
- The webhook waits up to 12 seconds for the result and returns it as the reply. If the job takes longer, the bot answers "reading..." and sends the result when it is ready.
- If Twilio refuses a message the bot sends on its own (free trial limit), the message is queued and delivered as the reply to the user's next message (for example "ok").

Data, security and cleanup

SnapDocc stores little, and it deletes what it stores. Everything lives in one SQLite file plus three working folders.

Store	Contents	Lifetime
`sessions` table	Phone number, photo path, state	Until the job ends, or expiry
`request_logs` table	Time, document type, format, engine, average confidence, seconds, status	Kept (used for the log viewer)
`edit_jobs` table	Random token, phone, extracted table, photo path	24 hours
`pending_deliveries` table	A message waiting for the user's next reply	Until delivered or cleaned
`data/uploads/`	Received photos	Deleted after direct jobs; old ones cleaned
`data/output/<random>/`	Generated files	Cleaned after about 25 hours

Security measures:

- Edit pages and file folders use random, unguessable names. Edit links expire after 24 hours.
- Excel and CSV cells that start with `=`, `+`, `-` or `@` are neutralised to stop formula injection.
- Images over 5 MB and files that are not images are rejected.
- Secrets (Twilio, Groq) live only in environment variables, never in the repository. `.env` is git-ignored.
- On the edit page, text is escaped before it is placed in the page.

Deployment, configuration and testing

The app runs in a Docker container on Render (free plan). Pushing to GitHub redeploys it automatically.

Variable	Purpose
`TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN`	Twilio login, also used to download photos
`TWILIO_WHATSAPP_NUMBER`	Fallback sender number
`GROQ_API_KEY`	Enables the Qwen vision reader

Variable	Purpose
`PUBLIC_BASE_URL`	The Render URL, used in file and edit links
`GROQ_MODEL` (optional)	Overrides the default model `qwen/qwen3.8-27b`

Deploy steps:

1. Push the project to GitHub (without `.env`).
2. Create a Docker web service on Render and add the variables above.
3. Set the Twilio sandbox webhook to `https://<service>.onrender.com/webhook` (POST).

Testing:

- `pytest` runs 85 tests. Twilio and Groq are mocked, so no network or keys are needed.
- `python -m app.ocr.groq_check photo.jpg` checks the AI reader on a real photo and prints the exact error if it fails.
- `python -m eval.evaluate` compares the OCR engines with CER and WER on sample images.
- `python -m app.db.view_logs` shows recent requests and summary statistics.

Known limitations and next steps

Limitation	Why	Fix
Rejoin the Twilio sandbox every 72 hours	Free sandbox rule	A paid or Meta-approved WhatsApp sender
Bot-sent messages need the user's "ok" reply	Free Twilio trial blocks them	Upgrade the Twilio account
Render free tier sleeps and loses local files on restart	Free plan	Keep-alive monitor, then a paid plan or external database
No accuracy figure on real handwriting	CER / WER script uses clean synthetic images only	Build a small set of real handwritten photos with ground truth
Very messy photos can be misread	Model limit	Yellow cells and the edit page catch it